Solution Requirements

Date	23 June 2026
Team ID	SWTID-2026-7169
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Fill out the descriptions and necessary details for each category provided below.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	*Users (farmers/researchers) access the FindYourCrop prediction tool directly through the web interface; no login is required for basic crop prediction, keeping the tool accessible to all users*	low
2	Authorization levels	*Single user role for this version — general user/farmer with access to Home, About, and FindYourCrop pages; no restricted/admin-only sections in the current scope*	low

3	External interfaces	*The system uses a locally trained ML model (no external APIs required for prediction); the Crop_recommendation.csv dataset was sourced from Kaggle during development*	medium

4	Transactions processing	*User submits soil/climate parameters via the FindYourCrop form (POST request) → Flask backend passes data to the trained model → prediction result returned and displayed on the same page*	High
5	Reporting	*The system displays the predicted crop recommendation directly on the FindYourCrop result page based on submitted input values*	Medium
6	Business rules, etc.	*Input parameters (N, P, K, temperature, humidity, pH, rainfall) must fall within realistic agricultural ranges; the model only recommends from the crop classes present in the training dataset*	Medium
7	Compliance to laws or regulations	*No personal or sensitive user data is collected or stored, minimizing data privacy risk; no specific regulatory framework (e.g., GDPR/HIPAA) applies given the non-personal nature of inputs*	low

8	*Other*	*System should allow easy retraining/updating of the ML model with new agricultural data in the future*	low

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	*Crop prediction should return results quickly after form submission, since the model is lightweight and runs locally*	*Prediction result returned in < 2 seconds*
2	Scalability	*System should handle increasing numbers of users and be extendable to larger datasets or additional ML models over time*	*Architecture supports scaling to cloud hosting (e.g., AWS/Heroku) without major redesign*
3	Security & Data Privacy	*Since no personal/sensitive data is collected, security focus is on protecting the trained model file and preventing malformed/malicious input*	*Input validation on all form fields; model file access restricted to server-side only*
4	Reliability & Availability	*Application should run reliably during local development/demo use, with plans for stable uptime once deployed to production hosting*	*Target 99% uptime once deployed to cloud hosting*
5	Usability & Accessibility	*Simple, clean, and intuitive interface so farmers with limited technical experience can easily input data and understand results*	*Form completed and prediction understood without external instructions*

6	*Other*	*Maintainability — code should be modular (separate model training, Flask app, and templates) so future developers can update/extend the system easily*	Clear separation of concerns across app.py, model.pkl, and HTML templates